# JS Strings

1. **What are Strings in Javascript?**
    a. The string is the collection of characters.
    b. Strings are immutable, means their content cannot be changed once created.
    c. However, you can create a new string based on an existing one.
    d. They are one of the most commonly used data types and can include letters, numbers, symbols, and even emojis.

2. **Ways to Create Strings:**
    a. **There are two ways to create a string in Javascript:**
        i. **Using String Literal:**
            1. By Using Single Quotes (' ')
            2. By Using Double Quotes (" ")
            3. By Using Template Literals (Backticks) (` `)

3. **By Using Template Literals (Backticks) (` `)**:

=> Template literals, enclosed by backticks ( `` ), allow for more flexible and advanced string handling. They support:

1. String interpolation
2. Multi-Line Strings

ii. **Using String Object:**

1. A string can also be created using the String() constructor, which creates an object representing a string.

2. **Syntax**:

```
let str = new String('Your text here');
```

3. **For Example →**

```
let strObj = new String("Hello World!");
console.log(strObj); // String {'Hello World!'}
console.log(typeof strObj); // object
```

4. **Important Note**: Using the String constructor creates an *object* , not a *primitive string*. This can sometimes lead to unexpected behavior, so it's generally *better to use string literals* unless you specifically need a String object.

3. **String length Property:**
   a. The *length* property is used to determine the number of characters in a string. This includes spaces, special characters, and punctuation.
   b. The length property is read-only and directly gives the total number of characters in the string.
   c. **Syntax**: string.length

## 4. Instance Methods of Strings:

### a. charAt():

i. Returns the character at a specified index (only positive index allowed).

ii. **Syntax**:

```
string.charAt(index)
```

iii. **Return Type**: String

iv. Nothing will logs if you passed negative values.

### b. charCodeAt():

i. Returns the *Unicode ASCII* value of the character at a specified index.

ii. **Syntax**:

```
string.charCodeAt(index)
```

iii. **Return Type**: Number

### c. concat():

i. Concatenates one or more strings and returns a new string.

ii. **Syntax**:

```
string.concat(str1, str2, ...)
```

iii. **Return Type**: String

**d. includes():**

i. Checks if a string contains a specified substring.

ii. **Syntax**:

```
string.includes(substring, startIndex)
```

iii. **Return Type**: Boolean

**e. indexOf():**

i. Returns the first occurrence index of a substring, or **-1** if not found.

ii. **Syntax**:

```
string.indexOf(substring, startIndex)
```

iii. **Return Type**: Number

**f. lastIndexOf():**

i. Returns the last occurrence index of a substring, or **-1** if not found.

ii. **Syntax**:

```
string.lastIndexOf(substring, startIndex)
```

iii. **Return Type**: Number

**g. slice():**

i. Extracts a section of a string and returns it as a new string.

ii. **Syntax**:

```
string.slice(startIndex, endIndex)
```

iii. **Return Type**: String

iv. slice() *allows* ***negative indexes*** *(counts from the end of the string).*

**h. split():**

i. Splits a string into an array based on a specified separator.

ii. **Syntax**:

```
string.split(separator, limit)
```

iii. **Return Type**: Array

**i. substring():**

i. Extracts a part of a string between two specified indexes.

ii. **Syntax**:

```
string.substring(startIndex, endIndex)
```

iii. **Return Type**: String

iv. substring() does **not** support negative indexes (treats them as 0).

**j. toUpperCase():**

i. Converts a string to uppercase.

ii. **Syntax**:

```
string.toUpperCase()
```

iii. **Return Type**: String

**k. toLowerCase():**

i. Converts a string to lowercase.

ii. **Syntax**:

```
string.toLowerCase()
```

iii. **Return Type**: String

**l. trim():**

i. Removes whitespace from both ends of a string.

ii. **Syntax**:

```
string.trim()
```

iii. **Return Type**: String

**m. trimStart():**

i. Removes whitespace from the beginning of a string.

ii. **Syntax:**

```
string.trimStart()
```

iii. **Return Type:** String

**n. trimEnd():**

i. Removes whitespace from the end of a string.

ii. **Syntax:**

```
string.trimEnd()
```

iii. **Return Type:**String

**o. startsWith():**

i. Checks if a string starts with a specified substring.

ii. **Syntax:**

```
string.startsWith(substring, startIndex)
```

iii. **Return Type:** Boolean

**p. endsWith():**

i. Checks if a string ends with a specified substring.

ii. **Syntax:**

```
string.endsWith(substring, length)
```

iii. **Return Type**: Boolean

**q. repeat():**

i. Repeats a string a specified number of times.

ii. **Syntax**:

```
string.repeat(count)
```

iii. **Return Type**: String

**r. padStart():**

i. Pads the start of a string with a specified character until it reaches the target length.

ii. **Syntax**:

```
string.padStart(targetLength, padString)
```

iii. **Return Type**: String

**s. padEnd():**

i. Pads the end of a string with a specified character until it reaches the target length.

ii. **Syntax**:

```
string.padEnd(targetLength, padString)
```

iii. **Return Type**: String

t. **replace():**

i. Replaces the first occurrence of a specified substring with another.

ii. **Syntax**:

```
string.replace(searchValue, newValue)
```

iii. **Return Type**: String

u. **replaceAll():**

i. Replaces all occurrences of a specified substring with another.

ii. **Syntax**:

```
string.replaceAll(searchValue, newValue)
```

iii. **Return Type**: String

v. **toString():**

i. Converts a string object to a primitive string.

ii. **Syntax**:

```
string.toString()
```

iii. **Return Type**: Primitive String

5. **Summary:**
    a. Basics of strings → length, indexing, accessing characters.
    b. Finding characters → `charAt`, `indexOf`, `lastIndexOf`.
    c. Checking content → `includes`, `startsWith`, `endsWith`.
    d. Extracting parts → `slice`, `substring`.
    e. Modifying → `replace`, `replaceAll`, `toUpperCase`, `toLowerCase`.
    f. Formatting → `trim`, `repeat`, `split`.